Tutorial 11: Unsupervised Machine Learning in R

BIOL90042

2026-10-14

Table of contents

Background . . . 1
Supervised vs unsupervised machine learning . . . 1
What is a k-means clustering algorithm? . . . 2
Study scenario and biological prior . . . 2
Why is normalisation essential for k-means? . . . 2
Setup . . . 2
Part 1: Prepare and explore the data . . . 2
Part 2: Build the k-means workflow . . . 3
Part 3: Fit the model and assign clusters . . . 3
Part 4: Visualise clusters vs known strains . . . 3
Part 5: Exploring different values of k . . . 3
Extension questions . . . 3
The Silhouette Score . . . 4
Save and apply on new datasets . . . 4
Answers . . . 4
Part 1: . . . 4
Parts 2 and 3: . . . 5
Part 4: . . . 6
Part 5: . . . 6
Extension questions: . . . 9

In this tutorial, learners should be able to:

1. Understand the difference between supervised and unsupervised machine learning and explore their applications in biomedical research.
2. Construct a complete `tidymodels` workflow (create a recipe, specify a mode, bundle a workflow) to train a k-means clustering algorithm with `tidyclust`
3. Fit a k-means model, inspect centroids, and assign cluster predictions to observations
4. Evaluate clusters against known biological groupings

Background

Supervised vs unsupervised machine learning

In Workshop 8 we developed **supervised** machine learning models in R using the `tidymodels` package (create a recipe, specify a model, bundle a workflow, and fit) which we can use when our data has labels: where we know what each grouping should be.

But what happens when we don't know this? This is when we use **unsupervised** machine learning, which aims to discover patterns or structures in the data **without prior knowledge of the groupings**. You have already encountered Multidimensional Scaling (MDS) in workshop, and you may have encountered Principal Components Analysis (PCA), k-means clustering or hierarchical clustering in your research so far.

In this tutorial, we reuse the same workflow concepts from Workshop 8 (recipe → model → workflow → fit), but apply them to an unsupervised task: k-means clustering with the `tidyclust` package.

For **supervised ML**, the pipeline included splitting our dataset into training and testing sets; our recipe contained both an outcome and predictor variable/s; we set the model (e.g., random forest), engine (R package) and mode (classification or regression); bundled the workflow; fit the model; and evaluated model performance.

For **unsupervised ML**, the pipeline differs in that we do not split our dataset into train/test sets, our recipe does not need an outcome variable (as we do NOT know the outcome), we set the model (e.g., k-means clustering) and engine, and do not need to set a mode.

We cannot explicitly evaluate model performance in the same way because there are no labels to measure against. Instead there are other, relative, measures we can employ, such as the **silhouette score**, which measures cluster *geometry* (how compact and well-separated the clusters are) rather than correctness. It is a way to compare results, like different values of k, not a statement that a clustering is "right": this still requires biological prior knowledge.

What is a k-means clustering algorithm?

K-means clustering algorithms are iterative and repeatedly ask: who belongs to each group, and where should each group's centre be? Repeating until stable leads us to the best possible solution.

A **centroid** is the point at the centre of a cluster: you can think of it as the cluster's "address". K-means has to start somewhere, so it places centroids at **random** positions and gradually moves them to better positions. Where they start affects where they end up: an unlucky random start can lead to a poor final clustering, even with the same data. To avoid relying on luck, we run the algorithm many times from different random starts and keep the best result, that is what `nstart = 25` does below.

Study scenario and biological prior

We apply unsupervised learning to the `noisy_mousezempic_dosage_data` dataset, which contains measurements for **344 mice** across three strains (CD1, BALBc, and Black6) housed across three cages, but this time we pretend we do *not* know the strain labels. Our aim is to cluster the mice based on physiological and dosage features only.

The best results will be obtained with prior knowledge of the optimal cluster number (`k`). Here we have a clear **biological prior**: there are **3 known strains** (CD1, BALBc, and Black6) giving us a strong reason to expect `k = 3` to be the most biologically meaningful solution, so we set `k = 3` directly in our model.

Why is normalisation essential for k-means?

K-means assigns observations to clusters based on **Euclidean distance**, the straight-line distance between points in feature space. If one variable is measured on a large scale (e.g., grams) and another on a small scale (e.g., millimetres), the large-scale variable will dominate the distance calculation, regardless of its biological relevance. Normalisation (mean = 0, standard deviation = 1) removes this scale bias.

Setup

Install the packages if you haven't already, then load the libraries and data:

```
install.packages(c(
  "tidyclust",
  "cluster",
  "factoextra"
))
```

```
library(tidyverse) # Data manipulation and visualisation
library(tidymodels) # Modelling framework
library(tidyclust) # Clustering extension for tidymodels
library(cluster) # Silhouette computation
library(factoextra) # Silhouette visualisation
library(patchwork) # Combining plots side by side

noisy_dosage <- read_csv(
  "https://raw.githubusercontent.com/WEHI-Education/BIOL90042_R_Course/refs/heads/main/data/noisy_mouse
)
```

Part 1: Prepare and explore the data

We explicitly keep only the numeric physiological features of our dataset and remove any missing values. Strain labels are deliberately **excluded**: we do not want to inform the algorithm about strain identity.

1. Create `dosage_clust` from `noisy_dosage`: remove all NAs with `drop_na()`, then `select()` only `initial_weight_g`, `tail_length_mm`, `drug_dose_g` and `weight_lost_g`.
2. Use `ggplot2` + `geom_point()` to plot `initial_weight_g` against `drug_dose_g`. Can you see any distinct groups?
3. Our dataset has four numeric variables. Modify the plotting code to explore different pairs of variables (i.e., 6 possible pairs total). Can you find the hidden groups in any of the two-dimensional plots?

Part 2: Build the k-means workflow

Just like in Workshop 8, we initialise our model with a recipe, a model specification, and a workflow bundle. The key difference: in unsupervised learning there is **no outcome variable**, so our recipe formula is `~ .` (no outcome, all predictors), and we do not set a mode. Because we removed all categorical variables, we also do not need `step_dummy()`.

Work through the following steps, using Workshop 8 as a reference:

1. **Recipe:** create `kmeans_recipe` with `recipe(~., data = dosage_clust)` and add `step_normalize(all_numeric_pr`
2. **Model:** create `kmeans_spec` with `k_means(num_clusters = 3)` and `set_engine("stats", nstart = 25)`.
3. **Workflow:** bundle them into `kmeans_workflow` with `add_recipe()` and `add_model()`.

Part 3: Fit the model and assign clusters

1. Set a seed (k-means starts from random centroids) and fit the workflow to `dosage_clust` with `fit()`.
2. Inspect the cluster centroids with `extract_centroids()`. Note the centroids are reported in **scaled units** because of `step_normalize()`.
3. Use `predict(final_kmeans_fit, new_data = dosage_clust)` to predict which cluster each mouse belongs to, and use `bind_cols()` to attach the predictions back to the data (call it `dosage_clustered`).
4. Use `left_join()` to join `noisy_dosage` to `dosage_clustered`, so we can compare against the known strain labels. Then cross-tabulate: `table(final_df$.pred_cluster, final_df$mouse_strain)`.

> **Discuss**
>
> - Does each cluster correspond to one strain? Which strain appears hardest for the algorithm to separate, and why might that be biologically?
> - What would a strong correspondence between clusters and strains, recovered without ever seeing the labels, tell us about the physiological features?

Part 4: Visualise clusters vs known strains

Compare the clusters to the known strains visually:

1. Create one scatter plot of `initial_weight_g` vs `drug_dose_g` coloured by `.pred_cluster`, and another coloured by `mouse_strain`.
2. Put them side by side with `patchwork` (e.g., `plot1 + plot2`). Are the two scatterplots similar? Are the groups clearly clustering or is there overlap/burring between the three strains/groups?

Part 5: Exploring different values of k

1. Refit the workflow with `num_clusters = 2`. (Hint: you only need to change one number in `kmeans_spec`, rebuild the workflow, and re-run `fit()` and `predict()`.) Re-make the cluster-coloured plot and the cross-tabulation. Which strains get merged?
2. Repeat with `num_clusters = 5`. What happens to the Black6 mice — do they stay together, or are they split apart?
3. Comparing your results for k = 2, 3 and 5: which k would you report, and what evidence supports your choice? When might a data-driven optimum disagree with a biological prior?
4. Inspect `extract_centroids()` for your k = 2 and k = 5 fits. Which variable changes the most between centroids as k increases? What does that tell you about how k-means "decides" to split a cluster?

Extension questions

Work through these in your own time. Answers are provided at the end of this handout.

The Silhouette Score

The silhouette score is a measure of how similar an object is to its own cluster compared to other clusters. It ranges from -1 to 1, where a higher value indicates that the object is well-matched to its own cluster and poorly matched to others.

The `fviz_nbclust()` function can calculate the silhouette score and plot various numbers of k clusters against the average silhouette width.

The silhouette score is computed from pairwise Euclidean distances between mice, the same distances k-means uses. Just as in our recipe, those distances must be computed on normalised data, otherwise the variable with the largest units dominates. Rather than scaling by hand, we extract exactly the normalised data the model saw from our fitted recipe using prep() + bake():

```
# Extract the scaled data from the fitted recipe
scaled_data <- kmeans_recipe %>%
  prep() %>%
  bake(new_data = dosage_clust)

# Try a range of k values and plot the average silhouette for each
fviz_nbclust(scaled_data, kmeans, method = "silhouette")
```

5. Use `fviz_nbclust()` on the scaled data extracted from your recipe (hint: `kmeans_recipe |> prep() |> bake(new_data = dosage_clust)`) with `method = "silhouette"`. Which k maximises the average silhouette width? Does it agree with your previous results?

Save and apply on new datasets

6. Save your fitted k = 3 model with `saveRDS(final_kmeans_fit, "final_kmeans_fit.rds")`. Then load the `new_mousezempic_dosage_data.csv` dataset, prepare it the same way (drop missing values, select the same four numeric variables), and use `predict()` to assign clusters to the new mice. Cross-tabulate the predictions against the new data's strain labels. *Caveat:* cluster labels are arbitrary, i.e., "Cluster_1" in your new predictions may not correspond to "Cluster_1" in training, so compare the *pattern* of the table, not the label names.

Answers

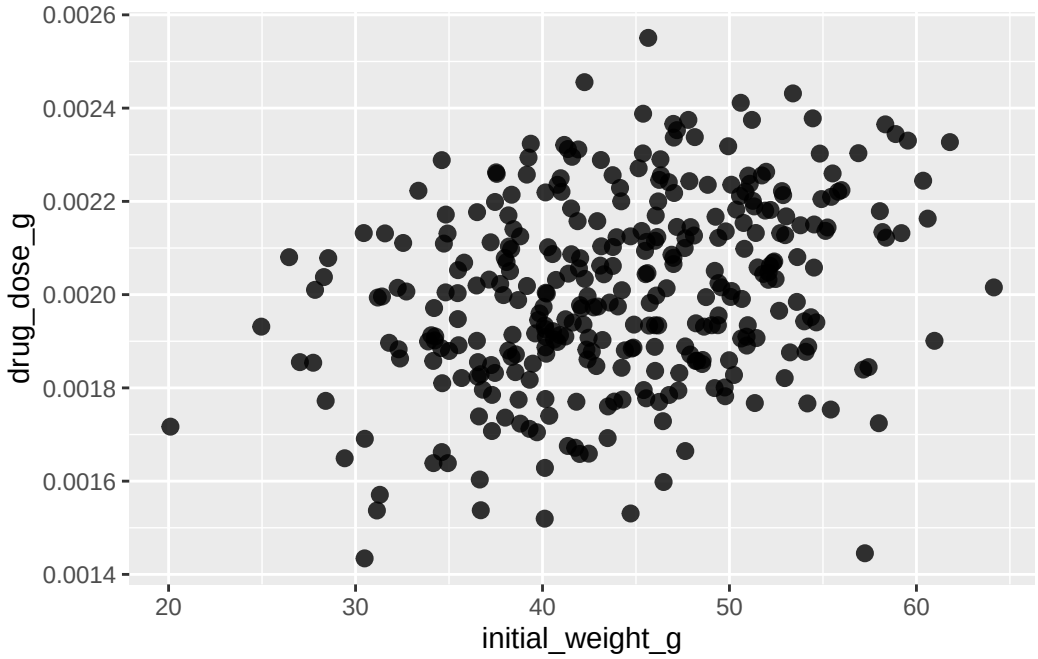
Part 1:

```
dosage_clust <- noisy_dosage %>%
  drop_na() %>%
  select(initial_weight_g, tail_length_mm, drug_dose_g, weight_lost_g)

nrow(dosage_clust)
```

```
[1] 333
```

```
dosage_clust %>%
  ggplot(aes(x = initial_weight_g, y = drug_dose_g)) +
  geom_point(size = 2.5, alpha = 0.8)
```



333 mice remain. No pair of variables shows clear, separated groups. Every two-dimensional plot looks like one large cloud. The groupings will only become apparent when all four variables are considered together, in the full four-dimensional feature space.

Parts 2 and 3:

```
kmeans_recipe <- recipe(~., data = dosage_clust) %>%
  step_normalize(all_numeric_predictors())

kmeans_spec <- k_means(num_clusters = 3) %>%
  set_engine("stats", nstart = 25)

kmeans_workflow <- workflow() %>%
  add_recipe(kmeans_recipe) %>%
  add_model(kmeans_spec)

set.seed(5947)
final_kmeans_fit <- kmeans_workflow %>%
  fit(data = dosage_clust)

extract_centroids(final_kmeans_fit)
```

```
# A tibble: 3 x 5
  .cluster  initial_weight_g tail_length_mm drug_dose_g weight_lost_g
  <fct>               <dbl>          <dbl>       <dbl>         <dbl>
1 Cluster_1          -0.923          0.148      -0.732       -0.769
2 Cluster_2           0.452          0.731      -0.181       -0.0467
3 Cluster_3           0.482         -0.804       0.871        0.785
```

```
dosage_clustered <- dosage_clust %>%
  bind_cols(predict(final_kmeans_fit, new_data = dosage_clust))

final_df <- noisy_dosage %>%
  left_join(
    dosage_clustered,
    by = c("weight_lost_g", "drug_dose_g", "tail_length_mm", "initial_weight_g")
  )

table(final_df$.pred_cluster, final_df$mouse_strain)
```

```
           BALBc Black6 CD1
  Cluster_1    15      3  94
  Cluster_2    45     11  49
  Cluster_3     8    105   3
```

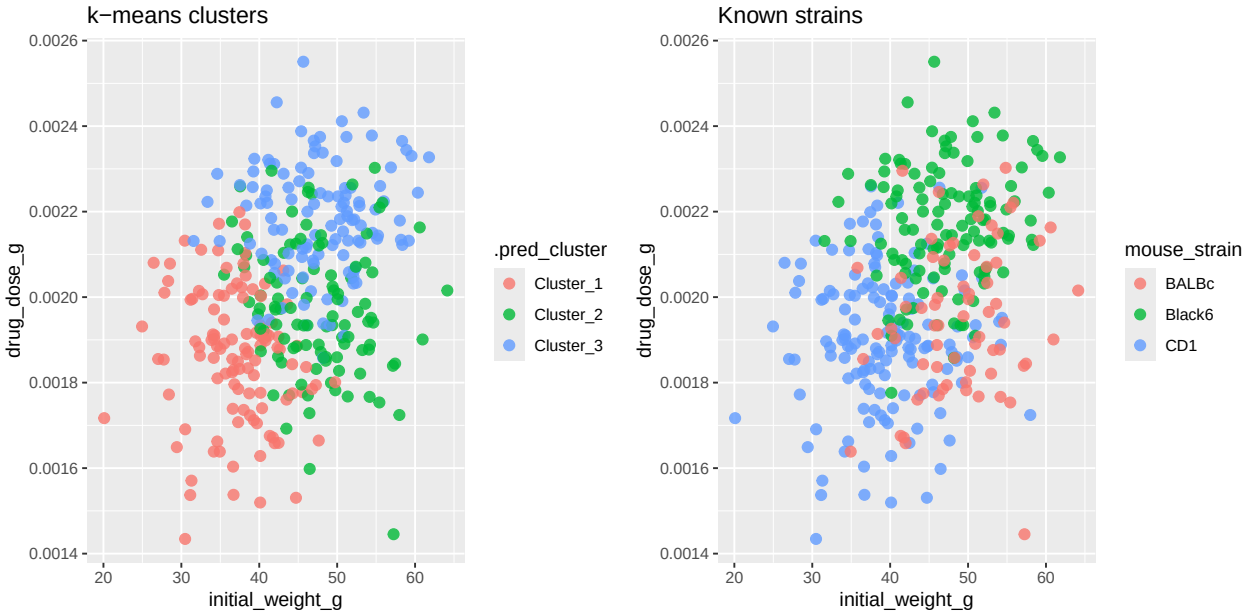
BALBc is the hardest strain to separate as it shares physiological overlap with CD1 (particularly in tail length), so it has the most misassignments. Black6 is typically well separated due to its distinctly higher body weight and drug dose. A strong cluster–strain correspondence confirms that the physiological features alone carry enough signal to distinguish the strains.

Part 4:

```
clusters_plot <- final_df %>%
  ggplot(aes(x = initial_weight_g, y = drug_dose_g, colour = .pred_cluster)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(title = "k-means clusters")

strains_plot <- final_df %>%
  ggplot(aes(x = initial_weight_g, y = drug_dose_g, colour = mouse_strain)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(title = "Known strains")

clusters_plot + strains_plot
```



Part 5:

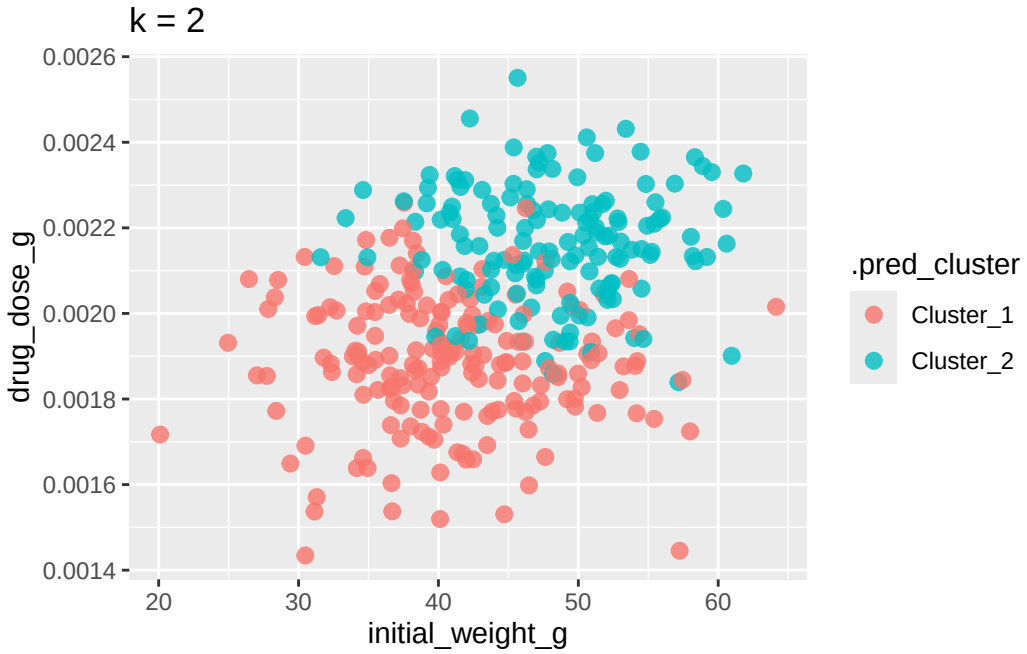
```
kmeans_spec_k2 <- k_means(num_clusters = 2) %>%
  set_engine("stats", nstart = 25)

kmeans_workflow_k2 <- workflow() %>%
  add_recipe(kmeans_recipe) %>%
  add_model(kmeans_spec_k2)

set.seed(5947)
fit_k2 <- kmeans_workflow_k2 %>%
  fit(data = dosage_clust)

k2_clust_df <- dosage_clust %>%
  bind_cols(predict(fit_k2, new_data = dosage_clust))
```

```r
k2_clust_df %>%
  ggplot(aes(x = initial_weight_g, y = drug_dose_g, colour = .pred_cluster)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(title = "k = 2")
```




```r
# join back to the full data to recover the strain labels
k2_noisy_dosage <- k2_clust_df %>%
  left_join(noisy_dosage)
table(k2_noisy_dosage$.pred_cluster, k2_noisy_dosage$mouse_strain)
```

```
           BALBc Black6 CD1
  Cluster_1    45      5 138
  Cluster_2    23    114   8
```

1. At $k = 2$, the cross-tabulation shows one cluster containing almost all Black6 mice (114 of 119, plus a handful of others), while the other cluster merges most CD1 mice (138 of 146) with a large share of BALBc mice (45 of 68). So **BALBc and CD1 are largely merged into a single cluster** and Black6 is separated: a worse *biological* solution than $k = 3$, even though it scores better *geometrically*.

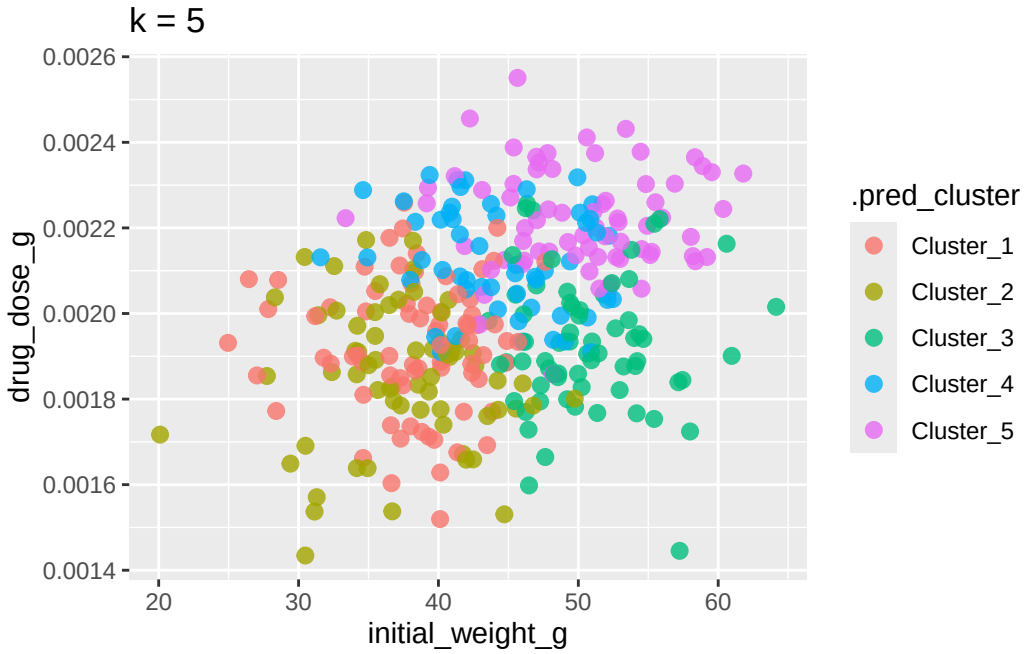
```r
kmeans_spec_k5 <- k_means(num_clusters = 5) %>%
  set_engine("stats", nstart = 25)

kmeans_workflow_k5 <- workflow() %>%
  add_recipe(kmeans_recipe) %>%
  add_model(kmeans_spec_k5)

set.seed(5947)
fit_k5 <- kmeans_workflow_k5 %>%
  fit(data = dosage_clust)

k5_clust_df <- dosage_clust %>%
  bind_cols(predict(fit_k5, new_data = dosage_clust))

k5_clust_df %>%
  ggplot(aes(x = initial_weight_g, y = drug_dose_g, colour = .pred_cluster)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(title = "k = 5")
```




```
# join back to the full data to recover the strain labels
k5_noisy_dosage <- k5_clust_df %>%
  left_join(noisy_dosage)
table(k5_noisy_dosage$.pred_cluster, k5_noisy_dosage$mouse_strain)
```

```
           BALBc Black6 CD1
  Cluster_1     8      5  63
  Cluster_2    12      2  51
  Cluster_3    37      3  26
  Cluster_4     6     45   4
  Cluster_5     5     64   2
```

2. At $k = 5$, the cross-tabulation shows the Black6 mice **split into two clusters** (45 in one, 64 in another), while the CD1 mice are also fragmented across three clusters and most BALBc mice (37 of 68) end up lumped together with CD1 mice in one cluster. Splitting strains apart like this adds no biological meaning — the algorithm is carving up natural within-strain variation.

3. $k = 3$ is the best choice: it matches the biological prior of three strains and the cross-tabulation recovers those strains without the under-splitting of $k = 2$ or the over-splitting of $k = 5$. This also illustrates when a data-driven optimum can disagree with a biological prior: when two true groups (here BALBc and CD1) overlap in feature space, geometric criteria such as the silhouette score prefer to merge them, even when merging is biologically wrong. The data-driven optimum should inform, not dictate, the choice of k.

```
# Centroids for k=2
extract_centroids(fit_k2)
```

```
# A tibble: 2 x 5
  .cluster  initial_weight_g tail_length_mm drug_dose_g weight_lost_g
  <fct>               <dbl>          <dbl>       <dbl>         <dbl>
1 Cluster_1          -0.415          0.413      -0.602        -0.536
2 Cluster_2           0.538         -0.535       0.781         0.695
```

```
# Centroids for k=5
extract_centroids(fit_k5)
```

```
# A tibble: 5 x 5
  .cluster  initial_weight_g tail_length_mm drug_dose_g weight_lost_g
  <fct>               <dbl>          <dbl>       <dbl>         <dbl>
1 Cluster_1          -0.764          1.02        -0.414       -0.0207
2 Cluster_2          -0.881         -0.294       -0.770       -1.05
3 Cluster_3           0.859          0.512       -0.397       -0.349
```

```
4 Cluster_4          0.0387         -1.30         0.586         0.181
5 Cluster_5          0.796          -0.288        1.06          1.17
```

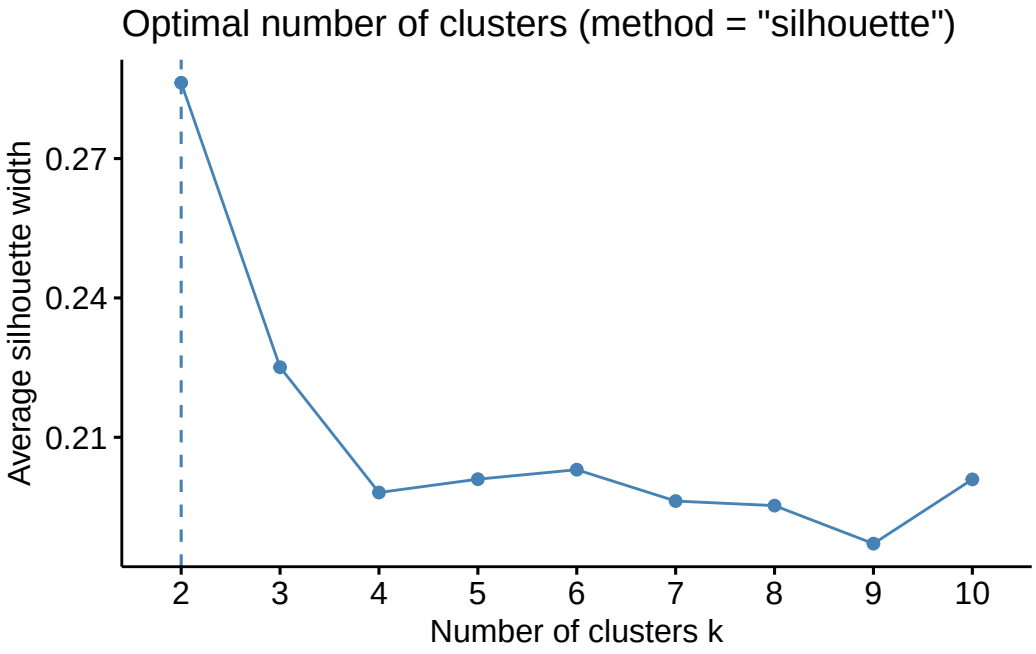
4. Comparing `extract_centroids()` for the k = 2 and k = 5 fits, the centroids separate most along `initial_weight_g`, `drug_dose_g` and `weight_lost_g` (e.g., at k = 5 these span roughly -0.9 to +0.9 in scaled units), while `tail_length_mm` varies less consistently. This shows that k-means splits clusters along whichever dimensions reduce within-cluster distances the most: it has no biological knowledge, it simply carves up feature space where points are most spread out.

Extension questions:

5. `fviz_nbclust(scaled_data, kmeans, method = "silhouette")` shows the average silhouette width is actually **maximised at k = 2**, not at our biological prior of k = 3. This is common and instructive: the silhouette criterion rewards well-separated clusters, so when two groups (here BALBc and CD1) lie close together in feature space it prefers to merge them — a worse *biological* solution even though it scores better *geometrically*. The silhouette score should inform, not dictate, the choice of k; with a strong biological prior we still select $k = 3$.

```r
scaled_data <- kmeans_recipe %>%
  prep() %>%
  bake(new_data = dosage_clust)

fviz_nbclust(scaled_data, kmeans, method = "silhouette")
```



6. Following the Workshop 8 pattern:

```r
saveRDS(final_kmeans_fit, "final_kmeans_fit.rds")
loaded_model <- readRDS("final_kmeans_fit.rds")

new_dosage <- read_csv("data/new_mousezempic_dosage_data.csv")

new_clust <- new_dosage %>%
  drop_na() %>%
  select(initial_weight_g, tail_length_mm, drug_dose_g, weight_lost_g)

new_predicted <- new_clust %>%
  bind_cols(predict(loaded_model, new_data = new_clust))

new_df <- new_dosage %>%
  left_join(
    new_predicted,
```

```
    by = c("weight_lost_g", "drug_dose_g", "tail_length_mm", "initial_weight_g")
  )

table(new_df$.pred_cluster, new_df$mouse_strain)
```

The cross-tabulation should show the same *pattern* as the training data (each cluster dominated by one strain), even though the "Cluster_1/2/3" labels themselves may be permuted, because cluster labels are assigned arbitrarily at each fit.